

Security Audit

Pruv Finance (Tokenization Platform)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Pruv Finance team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Pruv Finance is an RWA infrastructure layer bridging real-world assets (RWAs) and DeFi. It enables users to access trusted, tokenized assets that are freely tradable and fully composable across decentralized ecosystems.

Pruv streamlines the full lifecycle of real-world assets—from tokenization to distribution and DeFi integration—within a secure, compliant framework. The platform empowers issuers to tokenize assets such as real estate, commodities, and private equity, launch tokenized offerings, and manage them end-to-end.

For investors, Pruv provides seamless access to verified real-world assets that can be subscribed to, held, and redeemed through a transparent, regulated infrastructure. With built-in KYC/AML processes, encrypted transaction records, on-chain token verification, and robust custody solutions, Pruv ensures both compliance and trust at every step of the journey.

Project Name: Pruv Finance

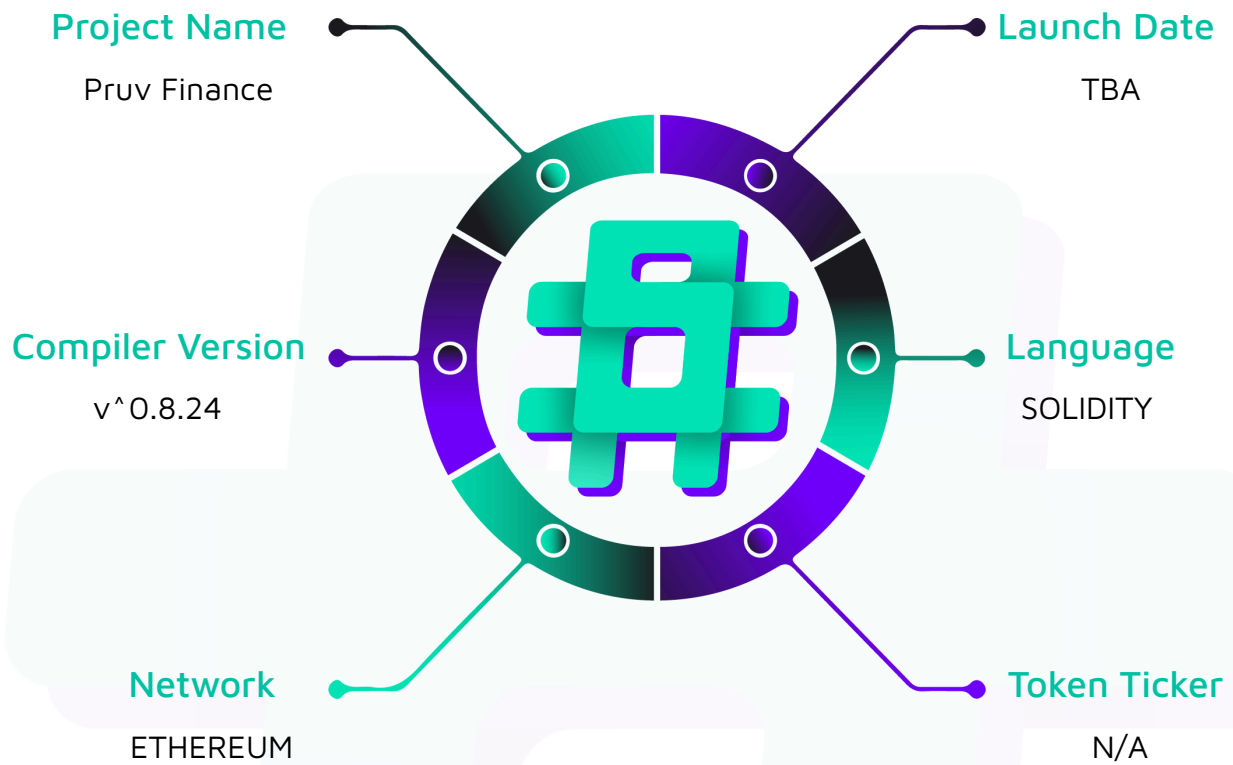
Project Type: Tokenization Platform

Compiler Version: ^0.8.24

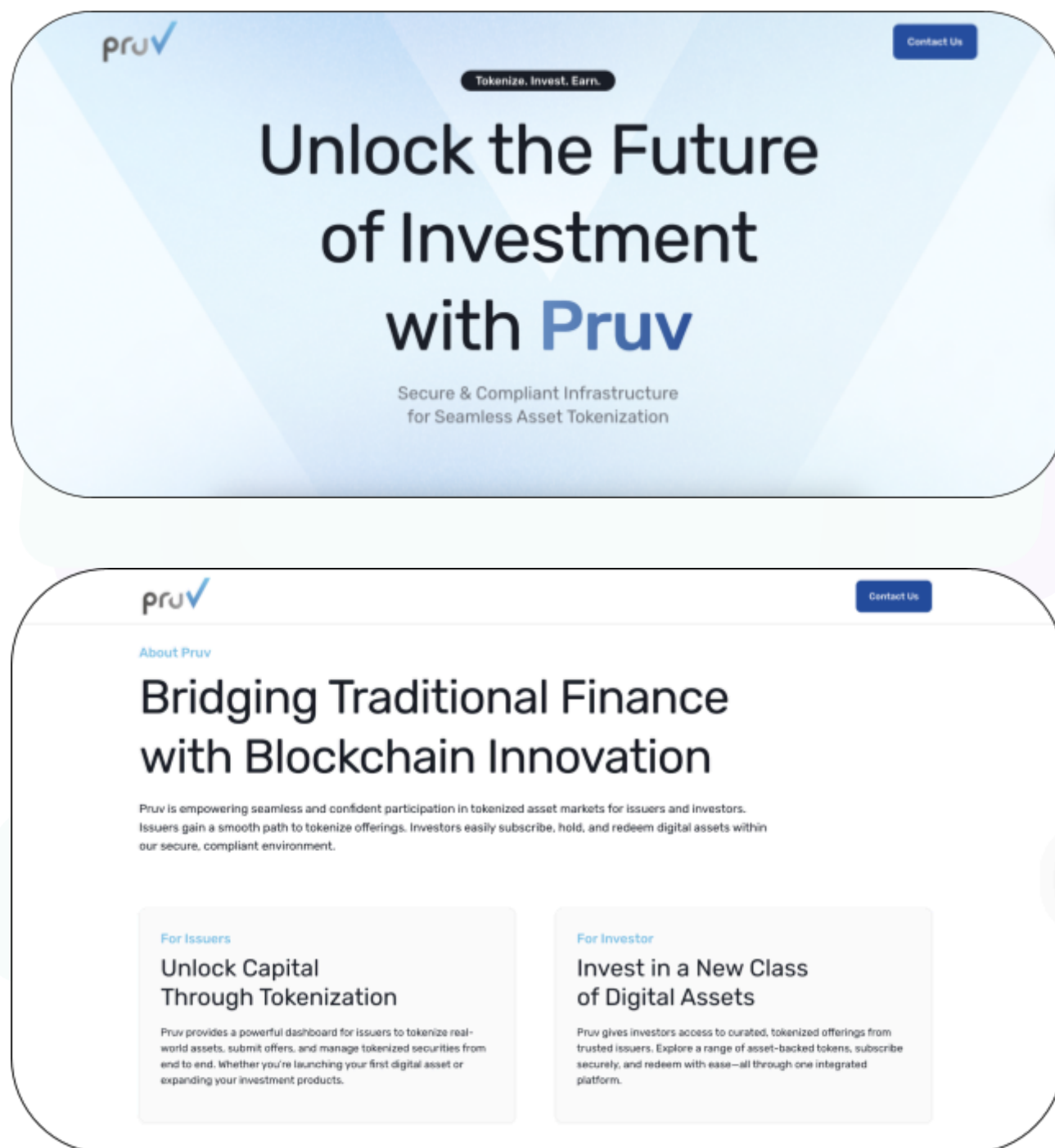
Website: <https://pruv.finance/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Pruv Finance project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Pruv Finance Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	Blacklist.sol
Contract 2	RWAConfigSTO.sol
Contract 3	RWAFactory.sol
Contract 4	RWAFee.sol
Contract 5	RWAFixedConversion.sol
Contract 6	RWAToken.sol
Contract 7	RWAValueFeeder.sol
Contract 8	Whitelist.sol
Audited GitHub Commit Hash	5f2d9378154756d482401b97e6d1bf1c1d96784b
Fix Review GitHub Commit Hash	e6938ea9be00b625094566c9fa90ea48d3b62865

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High severity vulnerability

7 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Blacklist.sol Soulbound ERC-1155 token that prevents transfers and can only be minted/burned via EIP-712 signatures. Used for blacklisting addresses in RWA compliance. Allows the owner to: - Set authorized signer for mint/burn operations	Contract achieves this functionality.
RWAConfigSTO.sol Compliance gate for ERC4626-style RWA vaults using optional whitelist/blacklist ERC-1155 NFTs. Enforces deposit/withdraw/transfer policies. Allows admin role to: - Set the whitelist and blacklist NFT contract addresses - Set per-address lifetime share deposit cap - Enable/disable withdrawals and transfers globally Allows the upgrader role to: - Authorize UUPS upgrades	Contract achieves this functionality.
RWAFactory.sol Factory that stores metadata and deploys modular RWA systems (config, conversion, token) as UUPS proxies using EIP-712 signatures Allows the owner to: - Set authorized EIP-712 signer - Set the default upgrader for deployed modules - Authorize UUPS upgrades	Contract achieves this functionality.
RWAFee.sol Fee management contract for RWA operations supporting percentage-based and fixed fees for entry and exit	Contract achieves this functionality.

operations. Allows admin role to: - Set fee type (percentage or fixed) and value for both entry and exit fees - Set the fee recipient address Allows the upgrader role to: - Authorize UUPS upgrades	
RWAFixedConversion.sol Fixed-price conversion module for RWA vaults providing deterministic asset-to-share and share-to-asset conversions. Allows set value role to: - Update the fixed conversion value Allows the upgrader role to: - Authorize UUPS upgrades	Contract achieves this functionality.
RWAToken.sol ERC4626 vault with modular compliance (config), pricing (conversion), and fee modules for RWA tokenization. Allows admin role to: - Set RWAConfig and RWAConversion module addresses - Set max supply limits - Admin withdraw assets (emergency/migration) - Admin burn user shares Allows the upgrader role to: - Authorize UUPS upgrades	Contract achieves this functionality.
RWAValueFeeder.sol Update RWA conversion values using EIP-712 signatures and access control. Allows the owner to: - Set authorized EIP-712 signer	Contract achieves this functionality.

- Authorize UUPS upgrades	
Whitelist.sol Soulbound ERC-1155 token similar to Blacklist, but used for whitelisting addresses in RWA compliance. Allows the owner to: - Set authorized signer for mint/burn operations	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Pruv Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Pruv Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Blacklist, Whitelist - Contract addresses can reject being blacklisted/whitelisted

Description

Contract addresses can prevent themselves from being blacklisted or whitelisted by implementing the `onERC1155Received` callback to revert or return an incorrect value, effectively bypassing the intended access control mechanism.

Vulnerability Details

Both the `mintWithSignature()` and `burnWithSignature()` functions in `Blacklist` and `Whitelist` contracts use OpenZeppelin ERC1155 functions, `_mint()` and `_burn()`, that internally call receiver callbacks `onERC1155Received()` on contract addresses.

The ERC1155 standard requires that when interacting with a contract address, the contract must implement `IERC1155Receiver` and return the correct magic value `onERC1155Received.selector` from the callback. If the callback reverts or returns an incorrect value, the entire operation fails.

This creates a vulnerability where a legitimate contract that does not implement `IERC1155Receiver` or a malicious contract that implements `onERC1155Received` to always revert or return an incorrect value, can prevent the contract from being blacklisted or whitelisted. The same applies to burning operations, preventing removal from the blacklist and whitelist.

Impact

Contract addresses can permanently avoid being blacklisted or whitelisted, breaking the intended access control mechanism and potentially allowing malicious contracts to operate without restrictions. This affects both adding addresses to the blacklist/whitelist (minting) and removing them (burning).

Recommendation

Given that the tokens are soulbound and cannot be transferred, the minimal modification approach is to override the `OpenZeppelin ERC1155 _updateWithAcceptanceCheck()` function to not perform the token acceptance check and not call `onERC1155Received()`. This single override will fix both minting and burning operations.

Status

Resolved

Low

[L-01] All contracts - Floating pragma

Description

All contracts use floating pragma statements `pragma solidity ^0.8.24`, instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and mainnet may pose unidentified security issues.

Recommendation

Consider locking the pragma version to the specific version `pragma solidity 0.8.24`.

Status

Resolved

[L-02] Blacklist#setApprovalForAll, Whitelist#setApprovalForAll - Missing override for setApprovalForAll()

Description

The `Blacklist` and `Whitelist` contracts implement soulbound tokens that are non-transferable, but they fail to override the `setApprovalForAll()` function, allowing users to set approvals for operators even though transfers are disabled.

Vulnerability Details

`Blacklist` and `Whitelist` implement soulbound ERC1155 tokens where transfers are completely disabled by overriding `safeTransferFrom()` and `safeBatchTransferFrom()` to always revert with `NonTransferable()`.

However, both contracts override transfer functions but leave `setApprovalForAll()` unimplemented, creating an inconsistent state where approvals can be set but are meaningless since transfers are disabled.

Impact

Users can set approvals for operators, creating confusion and an inconsistent state since these approvals serve no purpose when transfers are disabled.

Recommendation

Override the `setApprovalForAll` function in both contracts to revert with a custom error, maintaining consistency with the soulbound token design.

Status

Resolved

[L-03] Whitelist#setSigner - Missing SignerChanged event emission

Description

The Whitelist contract's `setSigner()` function does not emit a `SignerChanged` event when updating the authorized signer, unlike the Blacklist contract, which properly emits this event for transparency and off-chain monitoring.

Vulnerability Details

The Blacklist contract properly emits a `SignerChanged` event with both the old and new signer addresses, while the Whitelist contract lacks both the event definition and emission in the `setSigner()` function.

Impact

The missing `SignerChanged` event in the Whitelist contract reduces transparency and makes it difficult to track signer changes off-chain.

Recommendation

Add a `SignerChanged` event definition and emission to the Whitelist contract to match the Blacklist contract's behavior.

Status

Resolved

[L-04] RWConfigSTO#canDeposit - Missing global deposit disable mechanism

Description

The RWConfigSTO contract implements global disable mechanisms for withdrawals and transfers but lacks a similar mechanism for deposits, creating an inconsistency in access control and missing a potential security feature for emergency situations.

Vulnerability Details

The RWConfigSTO contract provides global control mechanisms for withdrawals and transfers that are validated in the `canWithdraw()` and `canTransfer()` functions. However, there is no equivalent `_depositEnabled` flag or `setDepositEnabled()` function to disable deposits.

Impact

The missing global deposit disable mechanism creates an inconsistency in access control design and removes a potential operational control that could be useful for maintenance.

Recommendation

Add a global deposit disable mechanism to match the existing withdrawal and transfer controls.

Status

Resolved

[L-05] RWAToken#maxDeposit/maxMint/maxWithdraw/maxRedeem - Missing overrides to account for compliance and supply limits per ERC-4626 specification

Description

The `RWAToken` contract does not override the `maxDeposit()`, `maxMint()`, `maxWithdraw()`, and `maxRedeem()` functions from `ERC4626Upgradeable`, failing to account for compliance checks (whitelist/blacklist) and supply limits, which violates the ERC-4626 specification requirements.

Vulnerability Details

According to the [ERC-4626 specification](#), the `maxDeposit()` function must factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily), it must return 0. The same principle applies to `maxMint()`, `maxWithdraw()`, and `maxRedeem()` functions.

The current implementation inherits from `ERC4626Upgradeable`, which provides default implementations. However, the `RWAToken` contract enforces several limits in its `_deposit()` and `_withdraw()` functions.

In `_deposit()` function:

- Global `maxSupply` limit
- Per-user compliance checks via `RWAConfigST0.canDeposit()`, which enforces whitelist/blacklist restrictions and per-user deposit limits

In `_withdraw()` function:

- Per-user compliance checks via `RWAConfigST0.canWithdraw()`, which enforces whitelist/blacklist restrictions and global withdrawal restrictions

However, the `RWAToken` contract does not override the four `max*()` functions from `ERC4626Upgradeable`, failing to account for these limits.

Impact

The missing overrides violate the `ERC-4626` specification and may cause integration issues with protocols that rely on these `max*()` functions to determine operation limits.

Recommendation

Override all four `max*()` functions to properly account for compliance checks and limits, or alternatively document that these functions are not `ERC-4626` compliant.

Status

Resolved

[L-06] RWAToken# - Missing setter function for the RWAFee module

Description

The RWAToken contract provides setter functions for RWAConfig and RWAConversion modules but lacks a corresponding setRWAFee() function, creating an inconsistency in the module management and limiting operational flexibility.

Vulnerability Details

The RWAToken contract implements three optional modules: RWAConfig, RWAConversion, and RWAFee. While RWAConfig and RWAConversion have setter functions to update the module address, RWAFee does not.

Impact

The missing setRWAFee() function creates an inconsistency in module management and limits operational flexibility for updating fee configurations.

Recommendation

Add a setRWAFee() function to match the pattern of the other module setters.

Status

Resolved

[L-07] RWAFee/RWAValueFeeder - Missing storage gaps for upgrade safety

Description

The `RWAFee` and `RWAValueFeeder` contracts are upgradeable but lack storage gap variables (`__gap`), creating an inconsistency with other upgradeable contracts in the codebase.

Vulnerability Details

The storage gap variables are useful for future variable additions, ensuring future upgrades' safety in case the contracts are used as base contracts of new ones. Including them is a good practice, but it's not a requirement. The contracts in the codebase should be consistent in this regard. However, there's an inconsistency where `RWAFee` and `RWAValueFeeder` lack storage gaps where others have them.

Impact

The missing storage gaps create an inconsistency in upgrade safety practices and may cause small complications during future upgrades when new state variables need to be added.

Recommendation

Add storage gap variables to both contracts to ensure upgrade safety and consistency.

Status

Resolved

QA

[Q-01]Blacklist#mint/mintBatch/burn/burnBatch, Whitelist#mint/mintBatch/burn/burnBatch - Unnecessary function definitions

Description

The contracts define public `mint()`, `mintBatch()`, `burn()`, and `burnBatch()` functions that only revert with custom errors, but these functions do not override any base contract functions and are not part of any standard.

Vulnerability Details

Both `Blacklist` and `Whitelist` contracts inherit from OpenZeppelin's ERC1155 contract, which only provides internal `_mint()`, `_mintBatch()`, `_burn()`, and `_burnBatch()` functions. The contracts define public versions of these functions that simply revert.

These functions are not overriding any base contract functions and are not part of the ERC-1155 standard. They serve no functional purpose and only add unnecessary code.

Impact

These unnecessary function definitions add code complexity without providing any functional value and may cause confusion about the contract's interface.

Recommendation

Remove the unnecessary `mint()`, `mintBatch()`, `burn()`, and `burnBatch()` function definitions from both contracts.

Status

Resolved

[Q-02] **RWAValueFeeder, RWAToken, Blacklist, Whitelist** - Unused declarations

Description

Several declarations are defined but never used in the codebase, including a constant, a custom error, and events that add unnecessary code complexity.

Vulnerability Details

The following declarations are defined but never used:

1. **RWAValueFeeder**: **DEFAULT_ADMIN_ROLE** constant is defined but never referenced in the contract
2. **RWAToken**: **InvalidAsset()** custom error is defined but never thrown
3. **Blacklist** and **Whitelist**: **SignedMintBatch** event is defined but never emitted

Impact

Unused declarations add unnecessary code complexity and may mislead developers about the contract's interface and functionality.

Recommendation

Remove the unused declarations.

Status

Resolved

[Q-03] RWAFixedConversion#convertToShares/convertToAssets -

Unnecessary decimal validation

Description

The functions validate that decimal values are `<= 77` to prevent overflow in `10 ** decimals` expressions, but this validation is redundant since Solidity 0.8.0+ has built-in overflow protection for arithmetic operations.

Vulnerability Details

Both `convertToShares()` and `convertToAssets()` functions include manual validation checks and followed by exponentiation operations:

```
if (shareDecimals > 77) revert InvalidDecimals();  
  
if (assetDecimals > 77) revert InvalidDecimals();  
  
uint256 shareUnit = 10 ** shareDecimals;  
  
uint256 assetUnit = 10 ** assetDecimals;
```

However, since Solidity 0.8.0+ automatically reverts on arithmetic overflow, the `10 ** decimals` operation will naturally revert if the result would overflow.

Impact

The unnecessary validation adds gas overhead for all function calls without providing additional safety benefits beyond Solidity's built-in overflow protection.

Recommendation

Remove the manual decimal validation checks and rely on Solidity's built-in overflow protection for the exponentiation operations.

Status

Resolved

[Q-04] RWAValueFeeder#_safeSetValue - Unnecessary try-catch complexity adds gas overhead and reduces error information

Description

The `_safeSetValue()` function uses try-catch to handle potential failures from `IRWAConversion.setValue()` calls, but this adds unnecessary complexity, gas overhead, and reduces error information compared to a direct call.

Vulnerability Details

The current implementation calls `IRWAConversion.setValue()` uses try-catch to handle potential failures and a generic error:

```
function _safeSetValue(
    address target,
    uint256 value
) internal returns (bool) {
    try IRWAConversion(target).setValue(value) {
        return true;
    } catch {
        return false;
    }
}

bool isSuccess = _safeSetValue(conversionAddress_, value_);

if (!isSuccess) {
    revert SetValueFailed();
}
```

However, the function could simply call `IRWAConversion(target).setValue(value)` directly, which would naturally revert with the original error if it fails.

Impact

The unnecessary try-catch complexity adds gas overhead, provides less informative error messages, making debugging more difficult, and adds unnecessary complexity.

Recommendation

Replace the `_safeSetValue()` function and success validation with a direct call to `IRWAConversion(target).setValue(value)`, which will naturally revert with the original error if the call fails, providing better error information and saving gas.

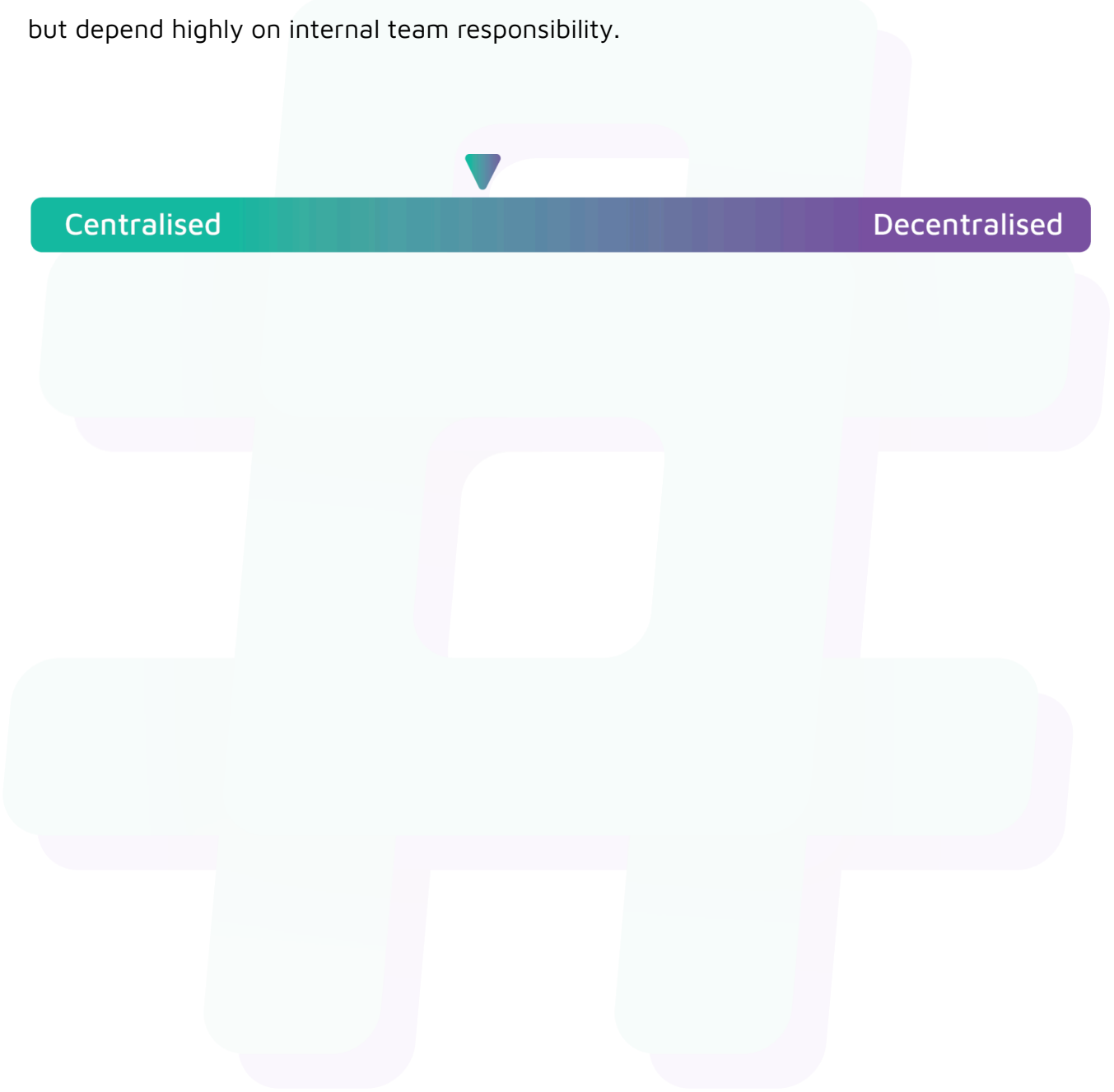
Status

Resolved

Centralisation

The Pruv Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Pruv Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.